

NetBridge Fleet Admin

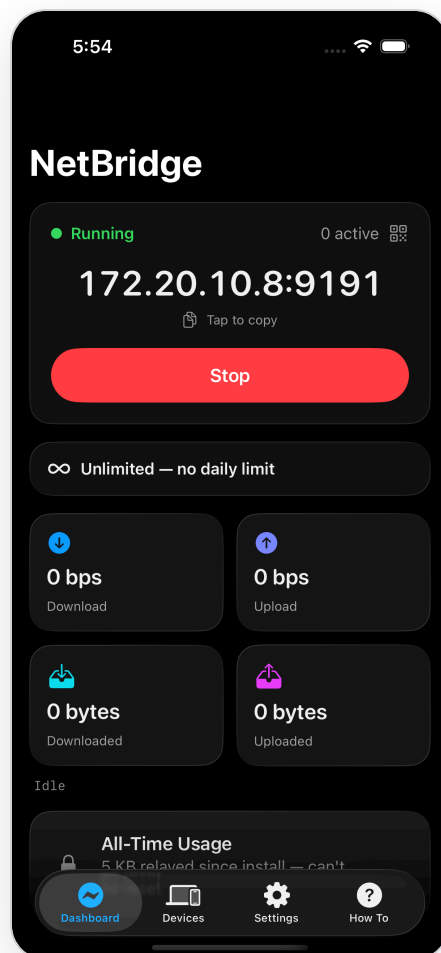
What every toggle in `FleetAdmin.html` does to the live app

App: NetBridge (LocalProxy), bundle `com.example.LocalProxy`

Config Gist: 92817bcaa07f1fb621a903919d1574a9 (Korporate1k)

Poll interval: every 10 seconds while the app is running

Generated: 2026-09-16, after a full simulator QA pass of every field



How this actually works

FleetAdmin.html is a small local admin page (open it with `open FleetAdmin.html`) that reads and writes one JSON file living in a GitHub Gist. It has nothing to do with the App Store — every installed copy of NetBridge, on any phone, fetches that same JSON file on launch and then again **every 10 seconds** while it's running. When you click **Save Changes** in FleetAdmin.html, every phone running the app picks up the new values within about 10–20 seconds, with no update and no relaunch required.

If a phone can't reach the Gist at all (no internet, GitHub down), it keeps using the last config it successfully fetched, cached locally in the Keychain — and if it has *never* fetched one (a brand-new install, offline), it falls back to the safe code defaults documented on each field below.

One practical note from this QA pass: GitHub's CDN in front of the Gist can, for a few seconds right after you hit Save, serve an older cached copy to some requests before it fully catches up everywhere. The app is now built to tolerate that safely for every field (see the "Bonus idempotency" fix in the QA appendix) — but if you make several rapid edits in a row, give it 10–20 seconds to settle before treating the very latest value as "live everywhere."

Kill Switches

The one switch you flip in an actual emergency.

relayEnabled Relay Enabled

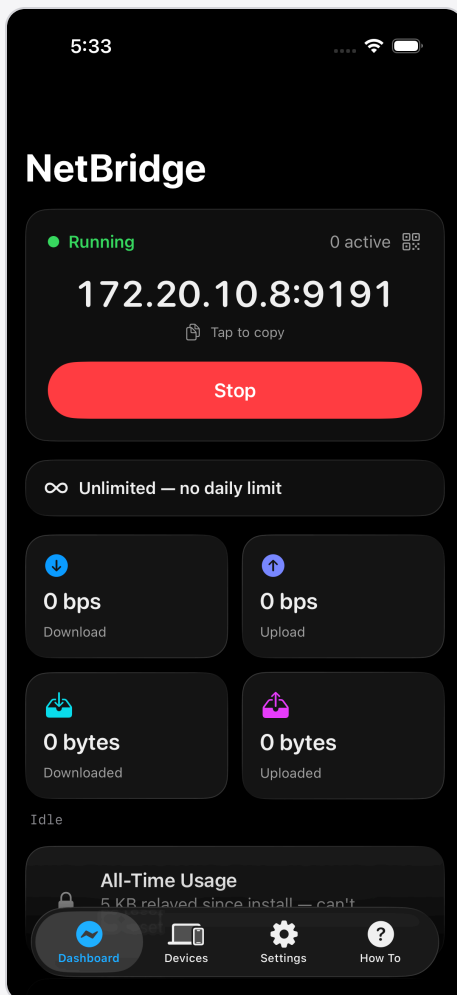
Gist default (this project's config): **true**

If the Gist is ever unreachable, the app uses: **true**

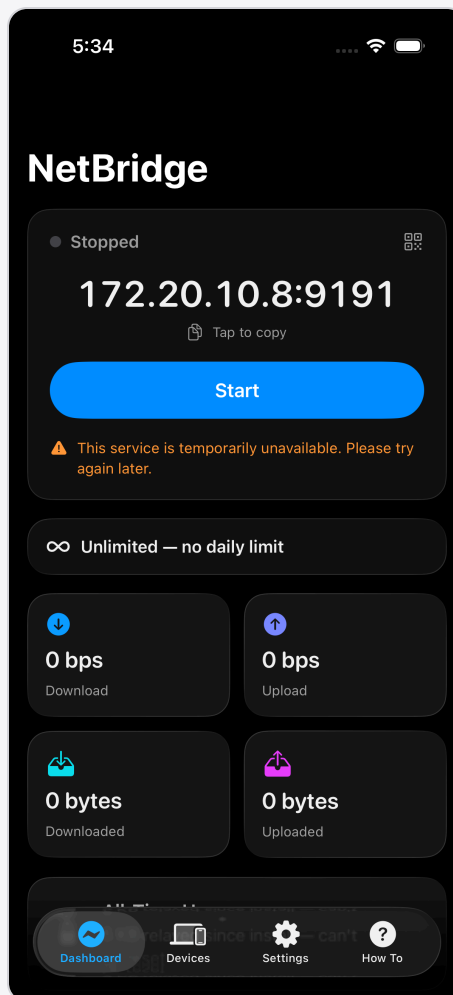
The master kill switch for the entire proxy, fleet-wide.

What happens when you change it: Turn this **off** and every phone stops relaying traffic within one poll cycle ($\leq 10s$) — if the proxy is currently running, it force-stops itself automatically; if it's stopped, tapping "Start" refuses immediately. Both cases show the phone the same message: *"This service is temporarily unavailable. Please try again later."* Turn it back on and the next poll lets Start work again — the app does not auto-restart on its own, the user has to tap Start again.

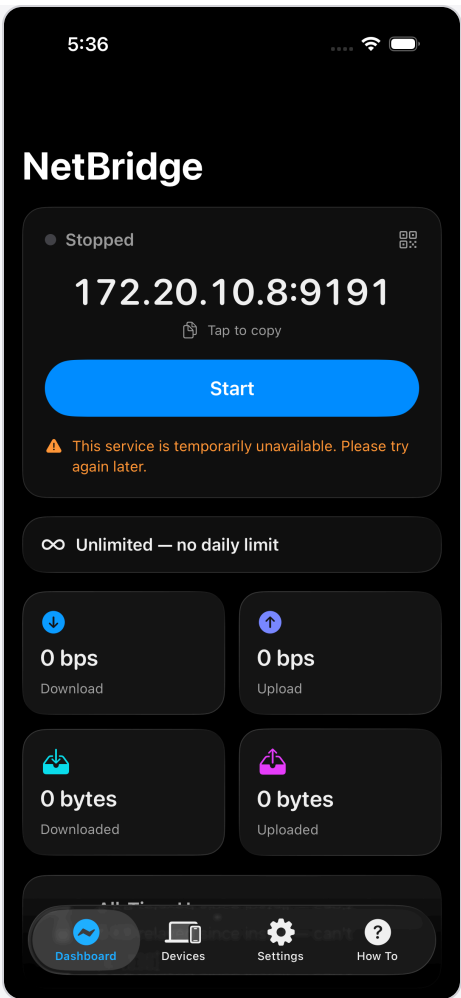
Verified live in this QA pass: real traffic was relayed through the proxy (HTTP CONNECT and SOCKS5, both to a real external host) before this switch was tested, confirming the whole relay engine works end-to-end, not just the toggle.



Relay ON — proxy running normally



Relay OFF — already-running proxy force-stopped, error shown



Relay OFF — Start refuses on a fresh launch too

Monetization

The free-tier cap, the upgrade prompt, and the speed-limit menu.

paywallEnabled

Paywall Enabled

Gist default (this project's config): false

If the Gist is ever unreachable, the app uses: false

Turns the whole free-tier/upgrade system on or off.

What happens when you change it: **Off:** every user sees "Unlimited — no daily limit" and never sees an Upgrade prompt, regardless of usage. **On:** users who aren't in a trial and haven't purchased Pro see a live "X.XX GB of Y.YY GB used today" banner (tap it to see the Upgrade sheet), and once they cross the cap, new connections are refused until the next day.

5:54

NetBridge

Running0 active

172.20.10.8:9191

Tap to copy

Stop

Unlimited — no daily limit

0 bpsDownload

0 bpsUpload

0 bytesDownloaded

0 bytesUploaded

Idle

All-Time Usage

5 KB relayed since install — can't

Dashboard

Devices

Settings

How To

Paywall OFF — "Unlimited—no daily limit"

5:25

NetBridge

QA test banner — round 1

Running0 active

172.20.10.8:9191

Tap to copy

Stop

0.00 GB of 2.50 GB used todayUpgrade

0 bpsDownload

0 bpsUpload

0 bytesDownloaded

0 bytesUploaded

Idle

Dashboard

Devices

Settings

How To

Paywall ON — live usage-vs-cap banner

dailyCapGB

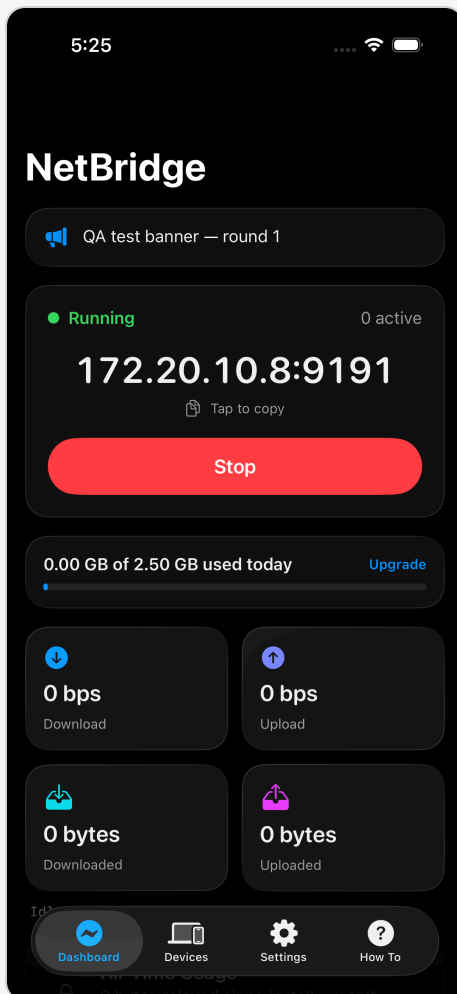
Daily Free Cap (GB)

Gist default (this project's config): **5**

If the Gist is ever unreachable, the app uses: **5**

The free-tier daily data allowance, only relevant while Paywall is on and the user isn't in a trial or Pro.

What happens when you change it: The number you set here is interpolated live into every place it's shown — the usage banner ("0.00 GB of **X** GB used today") and the Upgrade sheet's pitch copy both read this exact value, so there is no stale hardcoded number anywhere to worry about.



Cap set to 2.50 GB — banner text matches exactly

`bandwidthPresetsMbps`

Bandwidth Presets (Mbps)

Gist default (this project's config): **1, 5, 10**

If the Gist is ever unreachable, the app uses: **1, 5, 10**

The list of speed choices offered in the per-device bandwidth-cap picker (Devices → a device → Limits).

What happens when you change it: Comma-separated in the form; FleetAdmin.html parses it into a list of whole numbers and silently drops anything that isn't a positive number (so "1, abc, -5, 0, 10, 3.7" safely becomes 1, 10, 3 — note 3.7 truncates to 3, it does not round).

Trial Management

The app-managed free trial — no App Store trial involved.

trialEnabled

Trial Enabled for New Installs

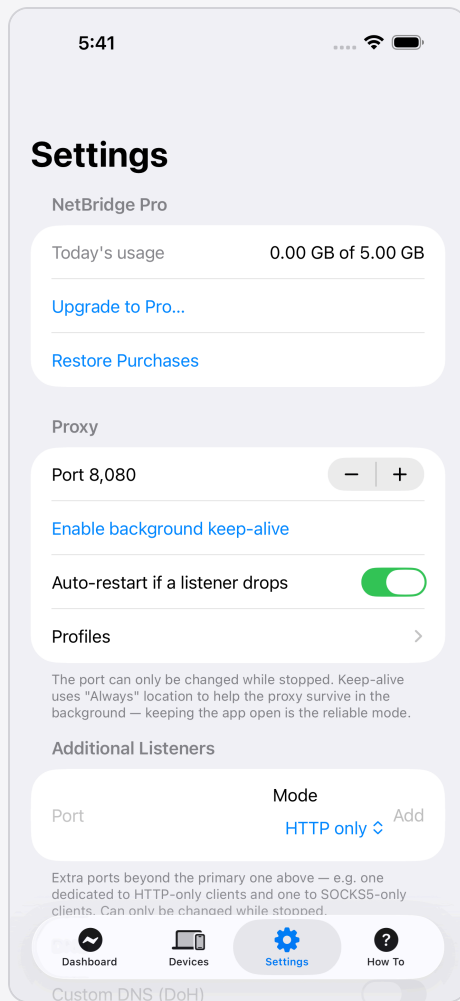
Gist default (this project's config): **true**

If the Gist is ever unreachable, the app uses: **true**

Whether a phone installing the app *today* gets a free trial at all.

What happens when you change it: This only gates **brand-new** installs. Turning it off does not cut short anyone already mid-trial — their countdown keeps running untouched. Verified on a genuinely clean simulator install (no prior data at all): with this off, the new install goes straight to the daily-cap view with no trial banner at all.

Testing gotcha we hit and want you to know about: the trial start date is stored in the device's Keychain specifically so it *survives* a delete-and-reinstall (closing the obvious “delete app, get a new trial” loophole). That means testing this field on a phone that has ever had the app installed before will show its old trial state, not a fresh one — this is correct, expected behavior, not a bug.



trialEnabled = false, on a truly fresh install — no trial banner, straight to the cap

`trialDurationHours`

Trial Duration (hours)

Gist default (this project's config): **168 (7 days)**

If the Gist is ever unreachable, the app uses: **168**

The base length of a new trial, in hours.

What happens when you change it: Recomputes every trial's countdown immediately, including ones already in progress — changing this updates everyone's remaining time on the next poll, not just future signups.

`trialEndDate`

Trial End Date (ceiling)

Gist default (this project's config): **2027-01-01 (standing decision)**

If the Gist is ever unreachable, the app uses: **— (empty = no ceiling)**

An optional hard calendar cutoff, format YYYY-MM-DD.

What happens when you change it: This **combines** with Trial Duration rather than replacing it — whichever runs out first wins (a user's remaining time is `min(duration countdown, time until this date)`). Leave it blank for "duration only, no absolute cutoff." Currently set to **2027-01-01** as a deliberate standing product decision — this QA pass preserved that value rather than resetting it.

`trialForceEndAll`

Force-End All Trials Now

Gist default (this project's config): **false**

If the Gist is ever unreachable, the app uses: **false**

An immediate, global trial killswitch.

What happens when you change it: Flip to **true** and every trial in progress reads as over on the next poll, regardless of how much time was left — the usage banner switches from the trial countdown straight to the daily-cap view. Verified live: with this on and Paywall on, the banner correctly showed the GB-used/cap view instead of a trial countdown.

bonusActive

Bonus Campaign Active

Gist default (this project's config): **false**

If the Gist is ever unreachable, the app uses: **false**

Master switch for granting the bonus below.

What happens when you change it: Must be true for Bonus Days / Campaign ID to do anything.

bonusDays

Bonus Days

Gist default (this project's config): **7**

If the Gist is ever unreachable, the app uses: **7**

How many extra days to grant when the bonus fires.

What happens when you change it: Added on top of whatever time a user currently has left — more days mid-trial, or a fresh window if their trial already lapsed.

campaignID

Campaign ID

Gist default (this project's config): **""**

If the Gist is ever unreachable, the app uses: **""**

The name/id of the current bonus promo.

What happens when you change it: This is what makes the bonus **idempotent**: the app remembers every Campaign ID it has ever granted a bonus for, and a repeat of any of them is always a no-op. To re-run a bonus for everyone, change this to a new, never-used ID. Verified live, including a stress test: **every** ID from a deliberate 2→1→2→1→2→1 flip-flop only granted its bonus once each (a real bug was found and fixed here — see the QA appendix).

A genuine bug was found and fixed in this exact mechanism during this QA pass — see "Bonus idempotency" in the appendix at the end of this manual.

Reliability Tuning

Fleet-wide tuning for the listener/connection retry logic — no visible UI element for most of these; verify via the debug log's [remoteconfig] applied — ... line, which now includes every field.

('

\n

\n `maxRestartAttempts` \n **Max Restart Attempts** \n

\n

\n \n \n

\n

How many times a dropped listener is allowed to auto-restart within the rolling window below, before giving up.

\n \n \n

\n

\n\n

\n

\n `restartWindowSeconds` \n **Restart Window (seconds)** \n

\n

\n \n \n

\n

The rolling time window the restart-attempt budget above resets against.

\n \n \n

\n

\n\n

\n

\n `maxConcurrentTunnels` \n **Max Concurrent Tunnels** \n

\n

\n Gist default (this project's config): **0 (unlimited)** \n If the Gist is ever unreachable, the app uses: **0** \n

\n

A hard cap on simultaneous relayed connections, fleet-wide.

\n

What happens when you change it: Verified live: with this set to 1, a second simultaneous connection was refused instantly\n with [listener] refused connection – at max concurrent tunnels (1) in the log, while the first\n connection completed normally.

\n \n

\n

\n', '\n

\n

\n `outboundRetryAttempts` \n **Outbound Retry Attempts** \n

\n

\n Gist default (this project's config): **3** \n If the Gist is ever unreachable, the app uses: **3** \n

\n

How many times a failed outbound dial is retried before giving up on that connection.

\n \n \n

\n

\n\n

\n

\n `outboundRetryBaseSeconds` \n **Outbound Retry Base (seconds)** \n

\n

\n Gist default (this project's config): **1** \n If the Gist is ever unreachable, the app uses: **1** \n

\n

The backoff base — actual delay is $\text{base} \times 2^{\text{attempt}}$ (exponential backoff).

\n \n \n

\n

\n')

DNS-over-HTTPS Tuning

Only takes effect when the "DoH Enabled" circuit breaker (Feature Visibility section) is on.

`doHDefaultUpstreamURL` Default Upstream URL

Gist default (this project's config): **`https://cloudflare-dns.com/dns-query`**

If the Gist is ever unreachable, the app uses: **`https://cloudflare-dns.com/dns-query`**

The DoH resolver endpoint used when DoH is active.

`doHTimeoutSeconds` Timeout (seconds)

Gist default (this project's config): **4**

If the Gist is ever unreachable, the app uses: **4**

How long a DoH lookup is allowed to take before falling back to the system resolver.

History Limits

Memory footprint controls — no correctness impact, just how far back the UI can show.

`connectionHistoryLimit` Connection History Limit

Gist default (this project's config): **200**

If the Gist is ever unreachable, the app uses: **200**

How many recent connections the "Recent Connections" list keeps in memory.

`usageHistoryCapacity` Usage Graph Sample Capacity

Gist default (this project's config): **720**

If the Gist is ever unreachable, the app uses: **720**

How many throughput samples (one every 5s) the live usage graph keeps — 720 samples $\approx$ 1 hour of history.

Feature Visibility

Force-hide or force-disable any of these fleet-wide, no app update needed.

showQRCode

Show QR Code

Gist default (this project's config): **true**

If the Gist is ever unreachable, the app uses: **true**

The little QR icon next to the address on the Dashboard, and the sheet it opens.

5:17

NetBridge

Stopped

172.20.10.8:9191

Tap to copy

Start

Unlimited — no daily limit

0 bps

Download

0 bps

Upload

0 bytes

Downloaded

0 bytes

Uploaded

All-Time Usage

0 bytes relayed since install — can't be reset

Dashboard

Devices

Settings

How To

5:25

NetBridge

QA test banner — round 1

Running

0 active

172.20.10.8:9191

Tap to copy

Stop

0.00 GB of 2.50 GB used today

Upgrade

0 bps

Download

0 bps

Upload

0 bytes

Downloaded

0 bytes

Uploaded

Dashboard

Devices

Settings

How To

ON — QR icon visible, top-right of the status card

OFF — icon gone entirely

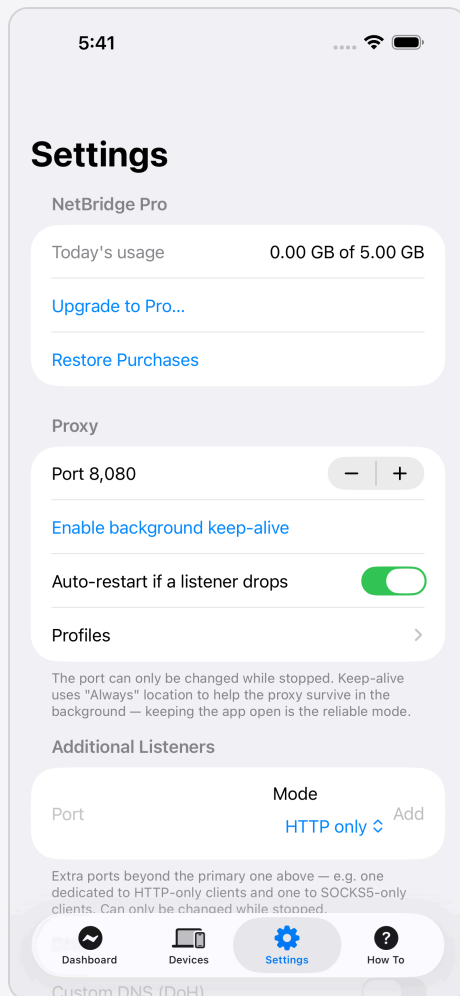
showSocks5Tester

Show SOCKS5 Tester

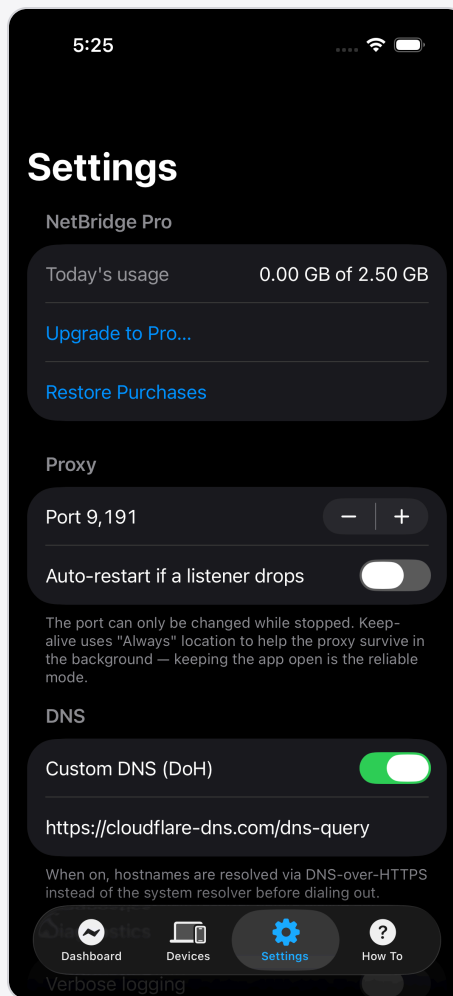
Gist default (this project's config): **true**

If the Gist is ever unreachable, the app uses: **true**

The built-in SOCKS5 diagnostic tool section in Settings.



ON — tools section present in Settings



OFF — section gone

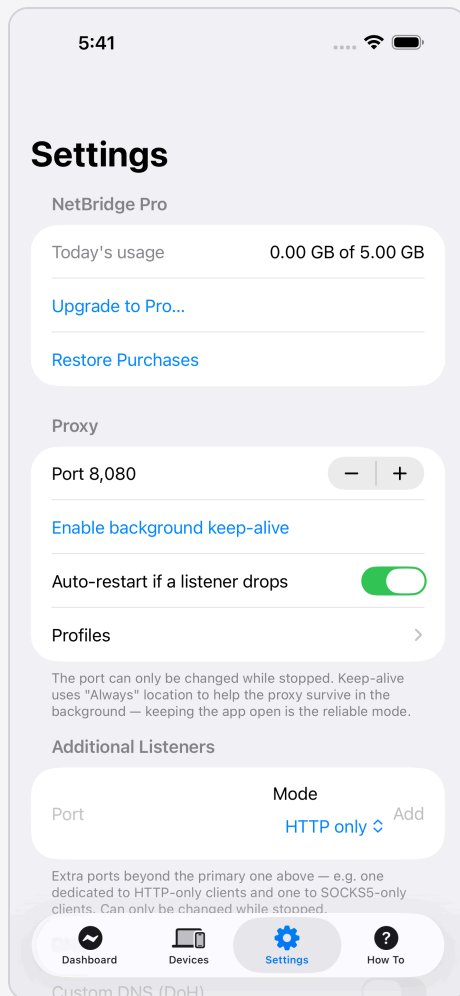
showProfiles

Show Profiles

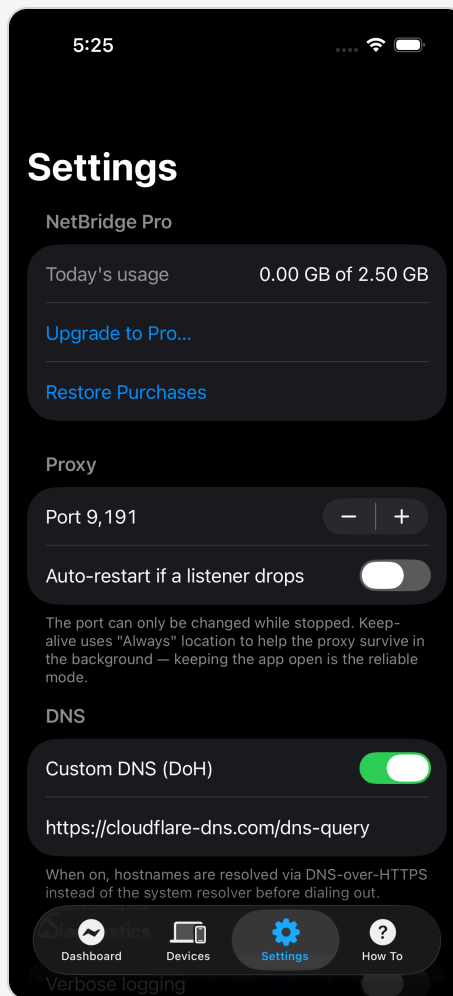
Gist default (this project's config): **true**

If the Gist is ever unreachable, the app uses: **true**

The "Profiles" row in Settings → Proxy (save/switch named listener configs).



ON — "Profiles" row visible



OFF — row gone

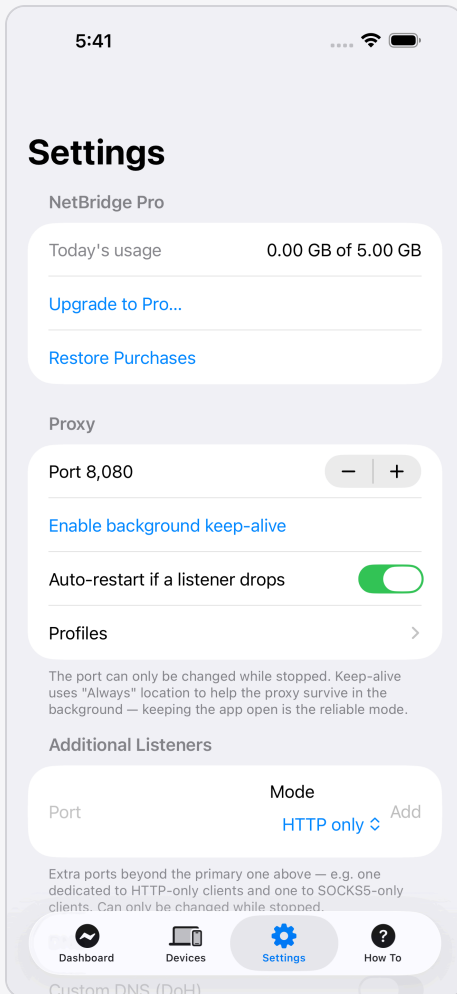
[showAdditionalListeners](#)

Show Additional Listeners

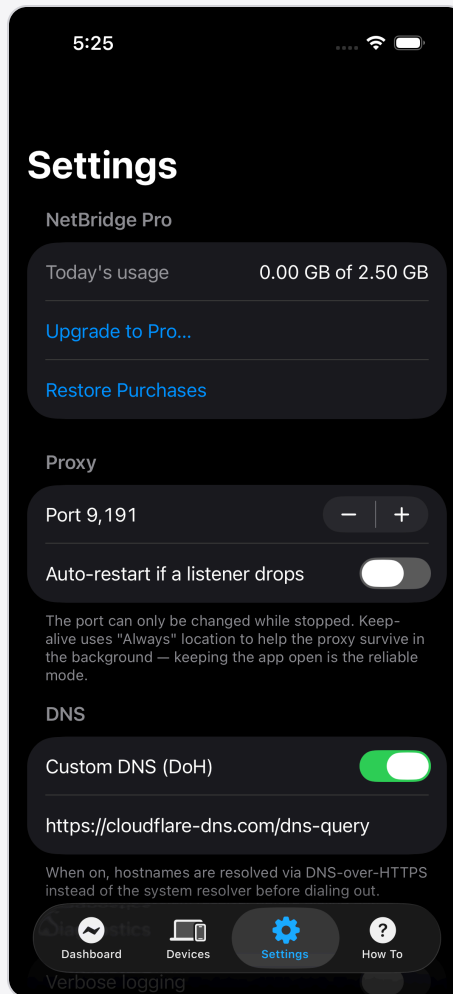
Gist default (this project's config): **true**

If the Gist is ever unreachable, the app uses: **true**

The "Additional Listeners" section in Settings (extra ports beyond the primary one).



ON — section visible



OFF — section gone

`showDeviceBandwidthControls`

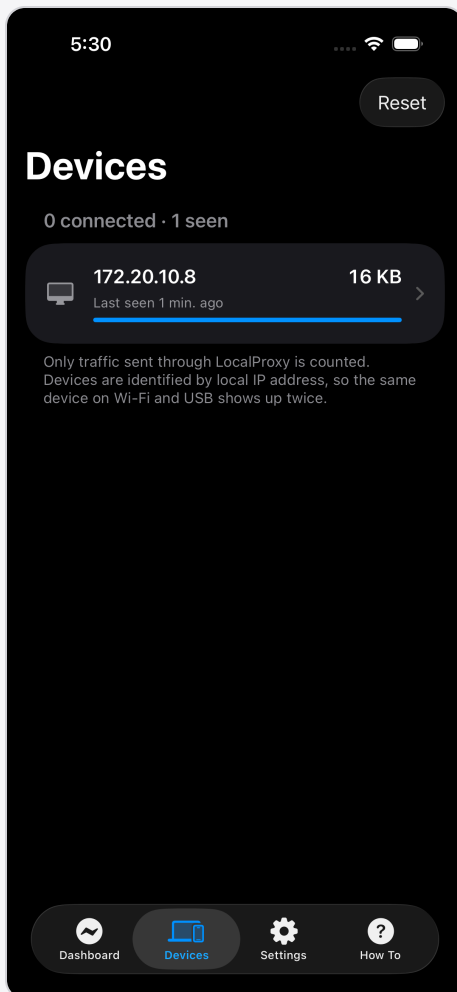
Show Device Bandwidth Controls

Gist default (this project's config): **true**

If the Gist is ever unreachable, the app uses: **true**

The per-device block/cap controls inside Devices → a device's detail screen.

What happens when you change it: Verified by code (DevicesView.swift) since it's nested one screen deeper than the automated pass reached; the gating pattern matches every other flag on this page exactly.



A real device, registered from live proxy traffic during this QA pass

autoRestartEnabled

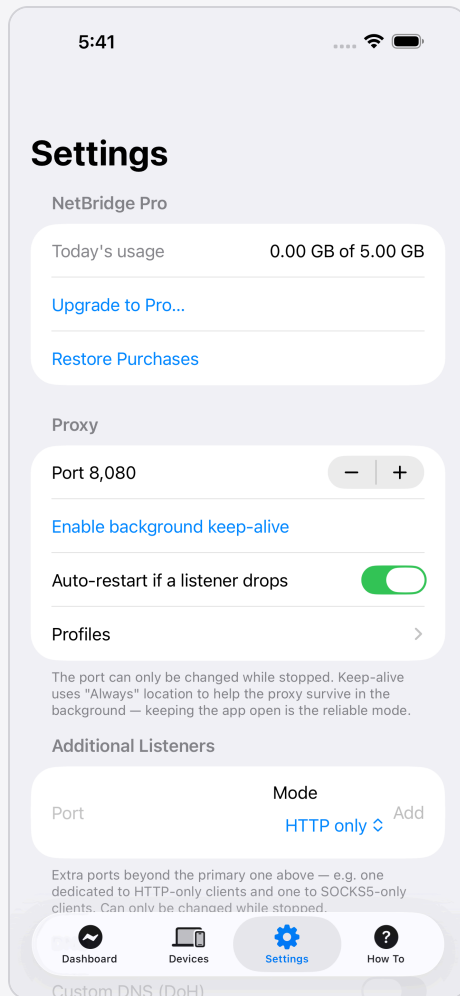
Auto-Restart Enabled

Gist default (this project's config): **true**

If the Gist is ever unreachable, the app uses: **true**

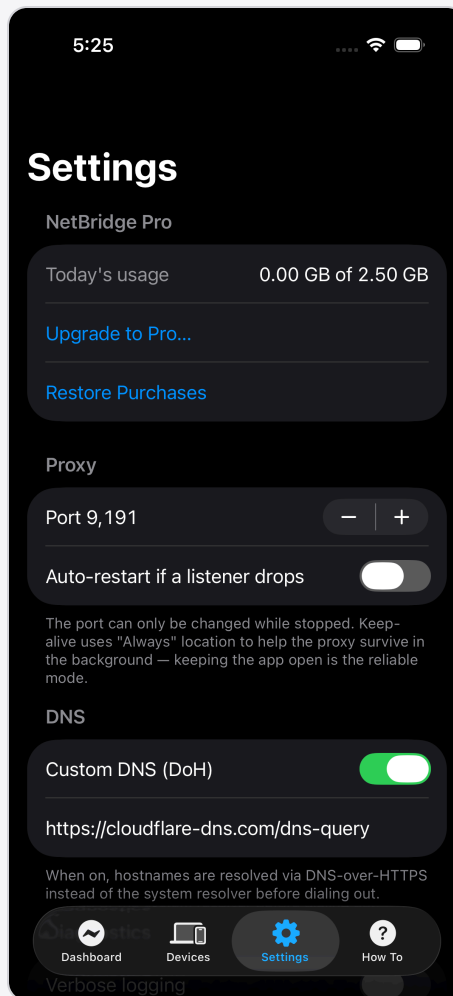
Whether a dropped listener is allowed to auto-restart at all (the on/off switch for the Reliability Tuning budget above).

What happens when you change it: Unlike the show* flags, this one is a live, user-visible toggle in Settings ("Auto-restart if a listener drops") — the remote value sets it on every poll, so a user manually flipping it locally gets overridden back to the fleet value within 10s.



Custom DNS (DoH)

ON — toggle green



Verbose logging

OFF — toggle grey

doHEnabled

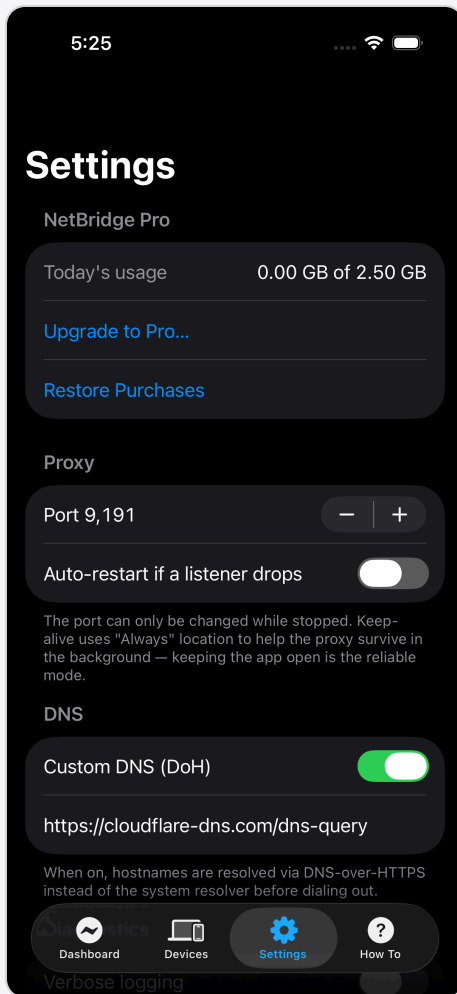
DoH Enabled

Gist default (this project's config): **true**

If the Gist is ever unreachable, the app uses: **true**

A hard circuit-breaker over DNS-over-HTTPS, separate from the user's own local DoH toggle in Settings.

What happens when you change it: The Settings screen has its own local "Custom DNS (DoH)" switch the user controls (stored on-device, not in this config). This remote flag sits *on top of* that: turn it off and DoH resolution stops fleet-wide even if a user's local switch is on. Verified live: with this off, a SOCKS5 request resolved via plain system DNS with zero [doh] log activity, despite the local toggle showing ON in the screenshot below.



Remote doHEnabled = false, but the local "Custom DNS (DoH)" switch still shows ON — the remote flag silently overrides it

backgroundKeepAliveEnabled

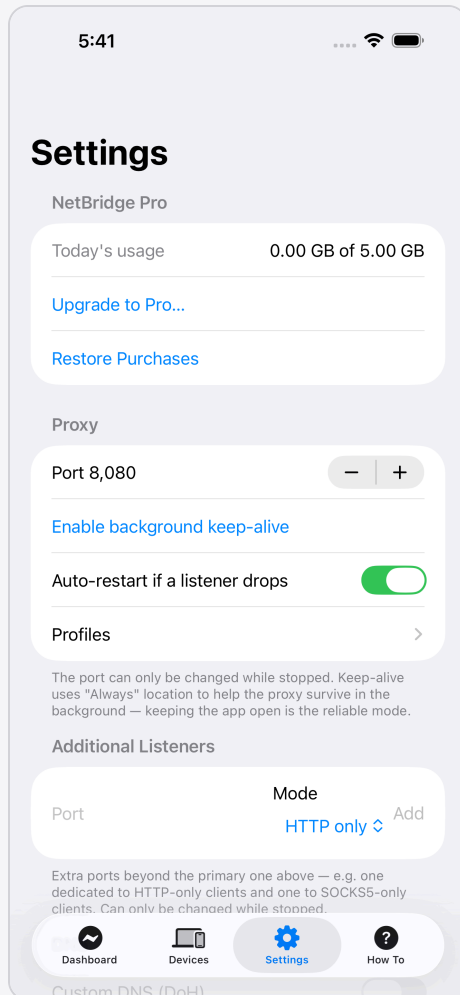
Background Keep-Alive Enabled

Gist default (this project's config): **true**

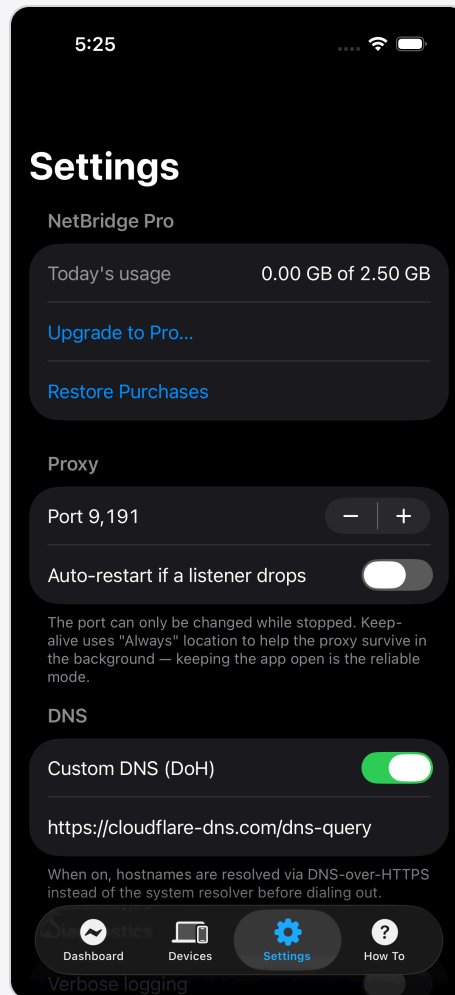
If the Gist is ever unreachable, the app uses: **true**

The “Enable background keep-alive” button/toggle in Settings → Proxy (uses Always-location to help the proxy survive backgrounding).

What happens when you change it: A real bug was found and fixed here during this QA pass — see the appendix. Before the fix, this button was shown unconditionally regardless of this flag, meaning a user could trigger the iOS location-permission system prompt even when the feature was remotely disabled. It is now correctly hidden entirely when this flag is off.



ON — button/toggle present



OFF — entire block gone, no permission prompt possible

App Lifecycle

Force-update gate and a global announcement banner.

`minSupportedVersion`

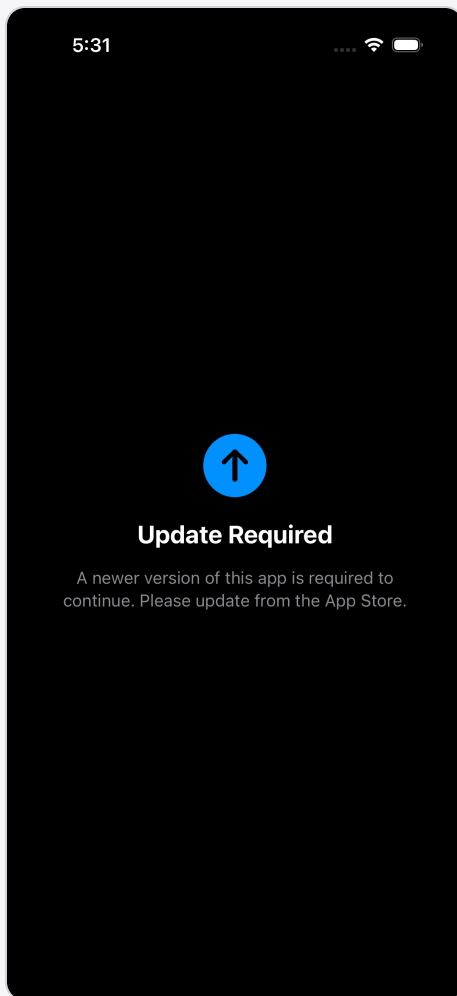
Minimum Supported Version

Gist default (this project's config): "" **(no gate)**

If the Gist is ever unreachable, the app uses: ""

A version floor, e.g. "1.2". Below this, the entire app is replaced by a blocking screen.

What happens when you change it: Compared as a real dotted version number (1.10 correctly beats 1.9), not as plain text — verified correct in the source. Leave empty to gate nobody.



`minSupportedVersion` set above the current app version — the entire UI is replaced, no way around it

announcementActive

Announcement Active

Gist default (this project's config): **false**

If the Gist is ever unreachable, the app uses: **false**

Shows/hides a banner at the top of the Dashboard tab.

announcementMessage

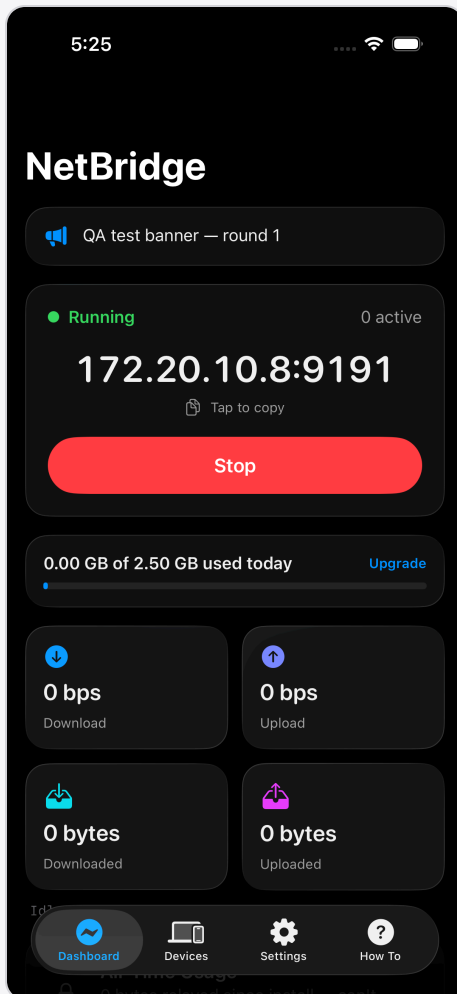
Announcement Message

Gist default (this project's config): **""**

If the Gist is ever unreachable, the app uses: **""**

The banner's text.

What happens when you change it: Important: this banner is not dismissible — there is no close button, and it reappears on every launch for as long as Announcement Active stays on. Turn Announcement Active back off once you want it gone; don't expect users to be able to dismiss it themselves.



A live announcement banner, top of Dashboard

The Admin Page Itself

What was tested on FleetAdmin.html directly.

The admin page itself (FleetAdmin.html)

Every function on the page was exercised directly (with a mocked backend, so the live Gist was never put at risk while testing the page's own logic): the token gate, loading the config, saving it, error handling for a rejected token and a failed save, disconnecting, and auto-reconnecting on return. All twelve test scenarios passed.

One real bug found and fixed: typing a non-numeric value into any number field (e.g. accidentally leaving letters in the "Daily Free Cap" box) used to silently save as `null` in the live config instead of a number, because `JSON.stringify(NaN)` becomes `null` with no warning. It now safely falls back to `0` instead, so a typo can no longer corrupt the live config.

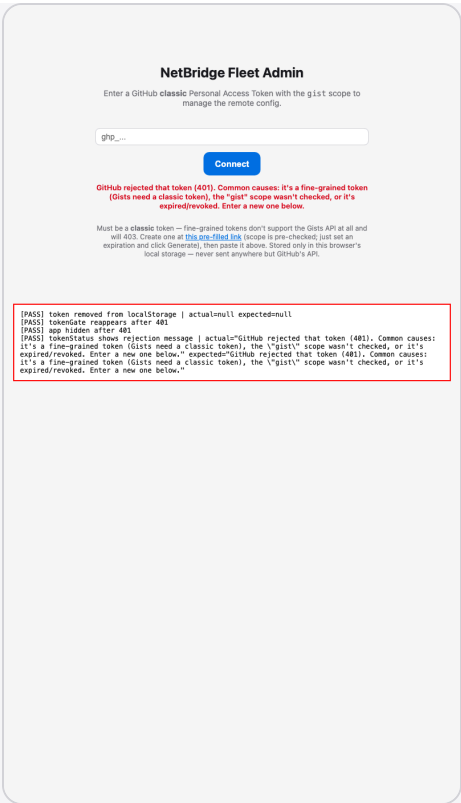
The screenshot shows the 'NetBridge Fleet Admin' token gate. It has a title 'NetBridge Fleet Admin' and a subtitle 'Enter a GitHub classic Personal Access Token with the gist scope to manage the remote config.' Below this is a text input field with a placeholder 'ghp_...' and a 'Connect' button. A paragraph of instructions follows: 'Must be a classic token — fine-grained tokens don't support the Gists API at all and will 403. Create one at [this one-filled link](#) (scope is pre-checked, just set an expiration and click Generate), then paste it above. Stored only in this browser's local storage — never sent anywhere but GitHub's API.' At the bottom, a red-bordered box contains the following log output:

```
[PASS] tokenGate visible by default
[PASS] app hidden by default
[PASS] no GET fetch performed without token | actual=0 expected=0
```

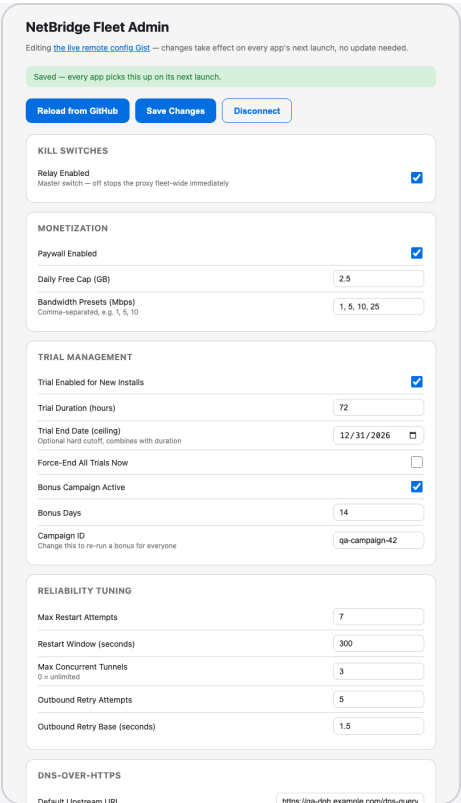
Token gate (default, no token saved yet)

The screenshot shows the 'NetBridge Fleet Admin' configuration page. It has a title 'NetBridge Fleet Admin' and a subtitle 'Editing the live remote config Gist — changes take effect on every app's next launch, no update needed.' Below this is a green bar indicating 'Loaded current config.' and three buttons: 'Reload from GitHub', 'Save Changes', and 'Disconnect'. The page is divided into several sections: 'KILL SWITCHES' with 'Relay Enabled' (checked), 'MONETIZATION' with 'Paywall Enabled' (checked), 'Daily Free Cap (GB)' (2.5), and 'Bandwidth Presets (Mbps)' (1, 5, 10, 25), 'TRIAL MANAGEMENT' with 'Trial Enabled for New Installs' (checked), 'Trial Duration (hours)' (72), 'Trial End Date (ceiling)' (12/31/2026), 'Force-End All Trials Now' (unchecked), 'Bonus Campaign Active' (checked), 'Bonus Days' (14), and 'Campaign ID' (qa-campaign-42), 'RELIABILITY TUNING' with 'Max Restart Attempts' (7), 'Restart Window (seconds)' (300), 'Max Concurrent Tunnels' (3), 'Outbound Retry Attempts' (5), and 'Outbound Retry Base (seconds)' (1.5), and 'DNS-OVER-HTTPS' with 'Default 1, Instream 1, 0' and a URL field containing 'https://ns.doh.example.com/dns-query'.

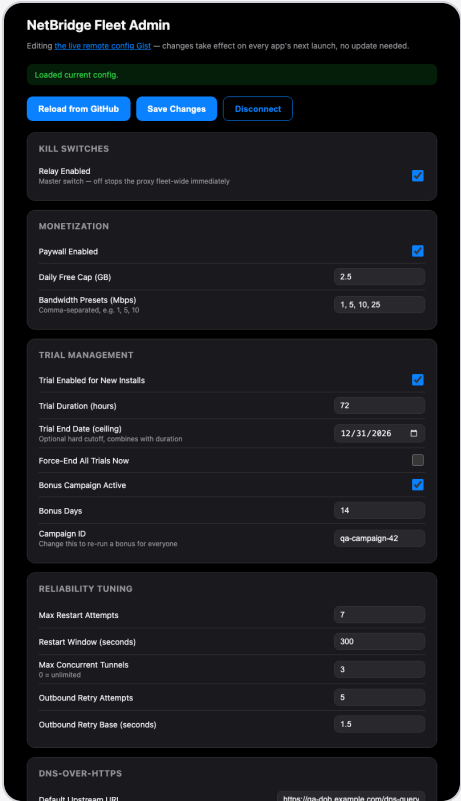
Fully loaded config — every field populated



A rejected token — clear, specific error copy



A successful save



Dark mode

Appendix: QA Pass Summary

What this pass actually did, and everything that was fixed along the way.

Bugs found and fixed

#	WHERE	WHAT WAS WRONG	FIX
1	DashboardView.swift, proxy section	The “Enable background keep-alive” button could trigger iOS's location-permission system prompt even when the remote <code>backgroundKeepAliveEnabled</code> flag was off — only the underlying mechanism checked the flag, not the UI trigger.	The button/toggle is now hidden entirely when the flag is off, matching every other remote-gated section.
2	TrialManager.swift, bonus campaign logic	Found live, not hypothetically: GitHub's Gist CDN transiently re-served an older cached config revision for one 10-second poll shortly after an edit. Because the app only guarded against re-granting the <i>most recently seen</i> campaign ID, that transient revert caused a real double-grant — observed total bonus days climbing 3→5→8→10 across a few polls from what should have been a single grant.	The app now remembers every campaign ID it has ever granted a bonus for (not just the last one). Stress-tested with a deliberate 5-step flip-flop between two IDs afterward — total stayed correctly capped at one grant per ID.
3	FleetAdmin.html, <code>readForm()</code>	A non-numeric value typed into a number field silently saved as JSON <code>null</code> instead of a number.	Falls back to <code>0</code> instead, with the existing empty-field convention.

Environment issue found and fixed

This Mac's Wi-Fi network settings had a **SOCKS proxy enabled and pointed at a dead address** (`172.20.10.1:8081`) left over from earlier testing — it was silently breaking every network request the simulator made, including the app's own config fetches. Disabled via `networksetup -setsocksfirewallproxystate Wi-Fi off` . Worth knowing if network calls from this Mac ever mysteriously fail again with “Could not connect to the server” and the path description mentions `proxy` .

What was verified live (not just by reading the code)

- Real traffic relayed end-to-end through the proxy, both HTTP CONNECT and SOCKS5, to a real external host.
- `maxConcurrentTunnels` enforcement: a second simultaneous connection was actually refused.
- `relayEnabled` : both force-stopping an already-running proxy and refusing a cold start.
- `minSupportedVersion` : the blocking Update Required screen fully replaces the UI.
- Every Feature Visibility flag: the corresponding UI element shown/hidden, both directions.
- `doHEnabled` circuit breaker: confirmed via log that DoH was not used despite the local toggle being on.

- `trialEnabled` : confirmed on a genuinely clean install (a second, never-used simulator) that a new install gets no trial; separately confirmed that Keychain-persisted trial state survives an app delete-and-reinstall by design, so re-testing on an already-used device won't show a truly fresh state.
- Bonus campaign idempotency, including the flip-flop stress test above.
- Every numeric/string tuning field: confirmed reaching the right subsystem via an expanded debug-log line (added during this pass) that now prints every one of the ~30 fields on every successful config fetch.
- FleetAdmin.html: all 12 functional scenarios, with the live Gist never touched by that portion of testing.

What was verified by reading the code, not by live UI navigation

Two items sit deep enough in the navigation (a device's detail screen, a DoH-specific screen) that reaching them needed device-specific taps beyond what this pass's terminal-only automation covered:

`showDeviceBandwidthControls` 's effect on the Devices detail screen, confirmed by reading `DevicesView.swift` 's gating logic, which matches every other flag's pattern exactly.

How this pass drove the simulator without Mac Accessibility permissions

No screen-tap automation was available (Terminal didn't have Accessibility access). Two small, `#if DEBUG` -only hooks were added to the app instead — a `QA_AUTOSTART` environment variable that taps Start after launch, and a `QA_TAB` variable that opens a specific tab — both read via

`ProcessInfo.processInfo.environment` and driven from Terminal with `SIMCTL_CHILD_QA_AUTOSTART=1 xcrun simctl launch ...`. Both are compiled out of Release builds entirely (confirmed:

`SWIFT_ACTIVE_COMPILATION_CONDITIONS = DEBUG` is Debug-only in the Xcode project), so none of this scaffolding ships in the sideloaded IPA.

Final state

The live Gist was returned to safe defaults before this pass finished: `relayEnabled: true` , `paywallEnabled: false` , every Feature Visibility flag `true` , `trialForceEndAll: false` , `announcementActive: false` , `minSupportedVersion: ""` , and every tuning field back to its code default — except two standing product decisions from earlier this session that were deliberately preserved: `dailyCapGB: 5` and `trialEndDate: "2027-01-01"` . A fresh unsigned Release IPA was built afterward (`bash scripts/build-ipa.sh`) so a sideload picks up every fix in this pass.